



# Analyse Technique : Assurance Qualité (test\_suite.py)

Ce document dissèque la suite de tests unitaires et d'intégration de **GeoMeteo**. Ce fichier constitue le "laboratoire" du projet, permettant de valider chaque composant de manière isolée et sécurisée avant toute mise en production.

## 1. Tests de l'Algorithme (Jacobson/Karn)

```
def test_jacobson_update_success(self):
    for _ in range(50):
        self.brain.update(observed_latency=0.1, success=True)
        self.assertLess(self.brain.get_timeout(), 5.0)
```



### Analyse Technique

On teste ici la convergence mathématique de l'algorithme. On simule 50 requêtes rapides pour observer si le timeout s'adapte à la baisse.



#### Pourquoi ce choix ?

- **Validation Scientifique** : Un algorithme prédictif ne peut pas être testé sur un seul appel. La boucle de 50 itérations prouve que le système "apprend" et que la moyenne mobile lissée (SRTT) fonctionne correctement.
- **Déterminisme** : Tester l'algorithme sans réseau garantit que la logique mathématique est pure et ne dépend d'aucun facteur externe aléatoire.

## 2. Isolation & Mocking (unittest.mock)

```
@patch('requests.Session.get')
def test_autocomplete_mock(self, mock_session_get):
    mock_resp = MagicMock()
    mock_resp.status_code = 200
    mock_session_get.return_value = mock_resp
```



### Analyse Technique

Nous utilisons des "doublures" (Mocks) pour intercepter les appels vers l'API OpenWeatherMap. Le décorateur `@patch` remplace la vraie connexion internet par un objet contrôlé.



#### Pourquoi ce choix ?

- **Indépendance Totale** : Tes tests peuvent tourner dans un avion, sans Wifi, ou même si

l'API OpenWeather est en panne.

- **Vitesse d'exécution** : Simuler une réponse JSON en mémoire prend quelques microsecondes, contre plusieurs centaines de millisecondes pour un vrai appel réseau. Cela permet de lancer des centaines de tests en un clin d'œil.

### 3. Gestion des "Unhappy Paths" (Erreurs)

```
def test_api_timeout_error(self, mock_session_get):
    mock_session_get.side_effect = requests.exceptions.Timeout("Trop long !")
    response = self.client.get("/get_weather?city=Paris")
    self.assertEqual(response.status_code, 504)
```

#### Analyse Technique

On ne teste pas seulement quand tout va bien. Ce bloc simule volontairement un crash réseau (side\_effect) pour vérifier la réaction du backend.

#### Pourquoi ce choix ?

- **Robustesse (Fail-Safe)** : Un bon développeur prévoit le pire. Nous vérifions ici que l'application ne renvoie pas une erreur brute illisible, mais qu'elle intercepte l'exception pour renvoyer un code HTTP sémantique (504) et un message clair à l'utilisateur.

### 4. Preuve Empirique de l'Optimisation (Cache)

```
def test_caching_mechanism(self, mock_session_get):
    self.client.get("/get_weather?city=Lyon")
    self.client.get("/get_weather?city=Lyon")
    self.assertEqual(mock_session_get.call_count, 1)
```

#### Analyse Technique

C'est le test le plus puissant. On appelle deux fois la même ville et on compte le nombre de fois où l'API a été réellement sollicitée via call\_count.

#### Pourquoi ce choix ?

- **Preuve de Performance** : Si call\_count est égal à 1 alors qu'on a fait 2 requêtes, cela prouve mathématiquement que ton système de cache LRU en mémoire fonctionne. C'est une garantie de réduction des coûts (quotas API) et de gain de vitesse.

### 5. Sécurité & Santé du Système

```
def test_security_headers(self):
    response = self.client.get("/")
    self.assertEqual(response.headers['X-Frame-Options'], 'DENY')
```

```
def test_health_check(self):  
    response = self.client.get("/health")  
    self.assertEqual(response.status_code, 200)
```

## **Analyse Technique**

On vérifie la présence des en-têtes de sécurité et la validité de la route /health utilisée par l'orchestrateur Render.

### **Pourquoi ce choix ?**

- **Conformité Production** : Ces tests garantissent que l'application respecte les standards de sécurité et que l'infrastructure de déploiement (le Health Check) ne rencontrera aucun problème lors du démarrage de l'instance.

*Ce fichier `test_suite.py` est le certificat de qualité du projet GeoMeteo, assurant une stabilité de niveau entreprise.*